

UNIVERSITATEA DE MEDICINĂ, FARMACIE, ȘTIINȚE ȘI
TEHNOLOGIE „GEORGE EMIL PALADE” DIN TÂRGU-MUREȘ
FACULTATEA DE INGINERIE ȘI TEHNOLOGIA INFORMAȚIEI
Programul de studii: Informatică (linia maghiară)



LUCRARE DE LICENȚĂ

Proiectarea unui sistem SmartLock bazat pe Bluetooth Low Energy și evaluarea empirică a algoritmilor de filtrare RSSI pe platforma ESP32

Student:

Koncz István-Róbert

Coordinator științific:

Conf. univ. dr. Finta Béla

Iulie 2026

MAROSVÁSÁRHELYI „GEORGE EMIL PALADE” ORVOSI,
GYÓGYSZERÉSZETI, TUDOMÁNYOS ÉS TECHNOLÓGIAI EGYETEM
MÉRNÖKI ÉS INFORMÁCIÓ-TECHNOLÓGIAI KAR

Tanulmányi program:
Informatika (magyar nyelven)



LICENSZ DOLGOZAT

Bluetooth Low Energy alapú okoszár rendszer tervezése és RSSI
szűrési algoritmusok empirikus értékelése ESP32 platformon

Hallgató:

Koncz István-Róbert

Tudományos koordinátor:

Conf. univ. dr. Finta Béla

2026 Július

Kivonat

Abstract

Tartalomjegyzék

Kivonat	1
Abstract	2
1. Bevezető	5
1.1. Okoszárok és BLE alapú hozzáférés-vezérlés	5
1.2. Kapcsolódó munkák	5
2. Tervezés	6
2.1. Az okoszárok rendszer általános architektúrája	6
2.2. A fizikai egység megtervezése	6
2.2.1. Funkcionális felépítés	7
2.2.2. Biztonsági és vészhelyzeti megfontolások	7
2.3. Az Android mobilalkalmazás tervezése	8
2.3.1. Felhasználói felület és funkcionális követelmények	8
2.3.2. Hozzáférés-kezelés és jogosultsági modell	8
2.3.3. Hibrid helymeghatározás: Geofencing és állapotgép	8
2.4. Kommunikációs modell és felhőalapú adattárolás	9
2.4.1. Aszinkron, adatbázis alapú vezérlés	9
2.4.2. A parancs és az állapot szétválasztása	9
2.4.3. Adatmodell és szerveroldali jogosultságkezelés	9
2.5. Közelségérzékelés (proximity sensing) és az RSSI-szűrés tervezési kérdései . . .	10
2.5.1. A nyers jel kezelésének tervezési követelményei	10
2.5.2. A szűrés eljárás mint nyitott kérdés	10
2.5.3. A küszöbérték testreszabhatósága	10
2.5.4. Tervezési korlát: az irány megkülönböztethetlensége	11
3. Felhasznált technológiák	12
3.1. Hardveres komponensek és perifériák	12
3.1.1. Az ESP32-S3-WROOM vezérlőegység	12
3.1.2. Tápellátás és feszültségszabályozás	13
3.1.3. Relé modul optocsatolóval	14
3.1.4. Szolenoidos zár és a védődióda	14
3.2. Bluetooth Low Energy és RSSI	15
3.2.1. A Bluetooth Low Energy működési elve	15

3.2.2.	Az RSSI mint közelségi mutató	16
3.2.3.	A NimBLE könyvtár	16
3.3.	A Firebase platform	16
3.3.1.	Firebase Authentication	16
3.3.2.	Firebase Realtime Database	17
3.4.	Az Android alkalmazás technológiái	17
3.4.1.	Kotlin és az Android keretrendszer	17
3.4.2.	Felhasznált könyvtárak és fejlesztői eszközök	17
4.	Megvalósítás	19
4.1.	A hardver összeállítása és bekötése	19
4.2.	Az ESP32 firmware	20
4.2.1.	Inicializálás és az eszköazonosító generálása	20
4.2.2.	Üzemmodok: aktivációs és normál működés	21
4.2.3.	BLE szkennelés és a jogosult beaconök azonosítása	21
4.2.4.	Az RSSI-szűrés megvalósítása	22
4.2.5.	Zárvezérlés és automatikus visszazárás	23
4.2.6.	Firebase-stream és a parancsfeldolgozás	23
4.3.	A Firebase adatbázis	23
4.3.1.	Az adatfa szerkezete	23
4.3.2.	Szerveroldali jogosultságkezelés	24
4.4.	Az Android alkalmazás	25
4.4.1.	A hibrid helymeghatározás megvalósítása	25
4.4.2.	A beacon sugárzása	25
4.4.3.	A felhasználói felület	26
5.	Eredmények	27
5.1.	A mérési módszertan	27
5.2.	Az RSSI-szűrők összehasonlítása	28
5.3.	Az éles rendszer használati mérése	29
5.3.1.	Nyitási sebesség: manuális vs. automatikus	29
5.3.2.	Téves nyitások vizsgálata	31
5.3.3.	Hosszú távú stabilitás	31
5.3.4.	A mérés korlátai	31
5.4.	Az eredmények értékelése	32
6.	Jövőbeli fejlesztések lehetősége és projekt összegzés	34
	Könyvészet	35
A.	Mellékletek	36

1. fejezet

Bevezető

1.1. Okoszárok és BLE alapú hozzáférés-vezérlés

1.2. Kapcsolódó munkák

2. fejezet

Tervezés

A tervezés alatt egyik fő szempont az volt, hogy a rendszer több, jól elkülöníthető részből álljon. Ennek köszönhetően a komponensek felelősségi köre átlátható, ami a fejlesztést is megkönnyíti.

2.1. Az okosvár rendszer általános architektúrája

Egy elosztott IoT (Internet of Things) architektúrára épülve, a rendszer a következő három fő részből áll:

- **Fizikai vezérlőegység:** felel a zárszerkezet fizikai működtetéséért, a hálózati kommunikációért, valamint a környezet Bluetooth alapú szkenneléséért.
- **Központi szerver:** Biztosítja a felhő alapú adatbázist, a kliens alkalmazás és a vezérlőegység között a valós idejű állapotszinkronizációt, kezeli a felhasználói jogosultságokat és a naplózást (logs).
- **Kliens alkalmazás:** Interfészt biztosít a felhasználó számára, lehetővé téve a záruk manuális vezérlését, a BLE alapú automatizáció beállítását, az okosvár regisztrálását és a hozzáférések kezelését.

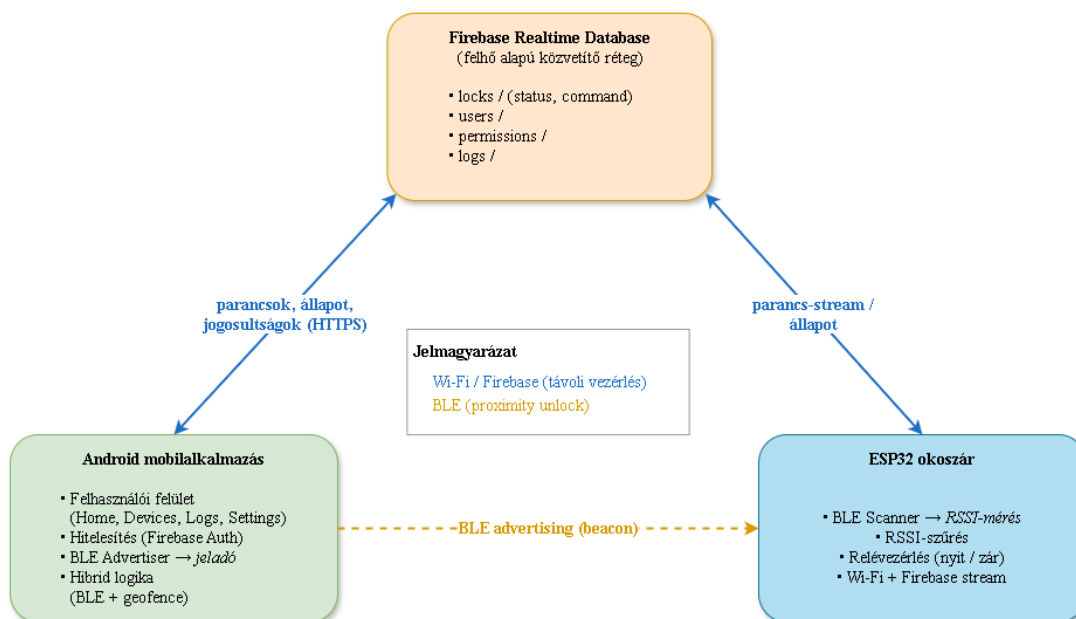
A három nagy komponens az 1. ábrán látható módon kapcsolódik. Két, eltérő jellegű csatornán valósul meg a kommunikáció:

- **Távoli vezérlés és állapotszinkronizáció:** Az adatbázison keresztül történik, emiatt a telefonnak és a zárnak nem kötelező ugyanazon a hálózaton tartózkodnia.
- **”Proximity” alapú automatikus nyitás:** A telefon és a mikrovezérlő közötti közvetlen BLE (Bluetooth Low Energy) kapcsolat szükséges hozzá.

A komponensek felépítését és a közöttük létrejövő kommunikációt a következő alfejezetek fejtik ki részletesen.

2.2. A fizikai egység megtervezése

Az okos zár hardveres központját egy ESP32-S3 WROOM mikrovezérlő adja. Ennek kiválasztását a beépített Wi-Fi és Bluetooth Low Energy (BLE) modulok indokolták, mivel, a rendszer műkö-



1. ábra. Az okoszárlat rendszer általános architektúrája

dése megkövetelte e két technológia egyidejű használatát. A tervezés során fontos szempont volt, hogy a fizikai egység minimális külső perifériával, költséghatékonyan, de megbízhatóan működjön.

2.2.1. Funkcionális felépítés

A vezérlőegység tervezéséhez a fő feladatkörök meghatározása szükséges. Az ezeket ellátó konkrét elektronikai modulokat a 3. fejezet mutatja be részletesen. Logikai szinten a vezérlőnek az alábbi alapvető feladatokat kell ellátnia:

- **Leválasztott kapcsolóelem:** A zárszerkezet nagyobb feszültségű áramkörét (power circuit) egy kapcsolón (switching element) keresztül kell vezérelni. Ez elektromosan leválasztja a vezérlő logikai (kisfeszültségű) oldalát a zár áramkörétől. Így a zár felőli oldalon keletkező zavarok és feszültségtűskék (transients) nem jutnak vissza a vezérlőbe.
- **Vizuális állapotjelzés:** A fizikai eszközöknél fontos a közvetlen felhasználói visszajelzés, ezért a tervben szerepel egy állapotjelző elem. Ennek feladata az eszköz pillanatnyi állapotának (például a nyitást vagy az aktivációs módot) vizuális megjelenítése.
- **Közvetlen, hálózathozfüggetlen felülbíráló bemenet:** A rendszernek tartalmaznia kell egy belső, manuális felülbírálást (override) biztosító bemenetet. Ez a felhőtől és a vezeték nélküli kapcsolatától függetlenül, közvetlenül a mikrovezérlőhöz kapcsolódik. Ennek biztonsági szerepét a következő alfejezet fejti ki.

2.2.2. Biztonsági és vészhelyzeti megfontolások

Egy beléptető rendszer tervezésekor kritikus szempont a hibátűrés és a biztonság. A fizikai nyomógomb integrálása biztonsági követelmény. Lehetővé kell tenni, hogy a helyiségben tartózkodó

dó személyek internetkimaradás, serverhiba, vagy a telefon lemerülése esetén is akadálytalanul kinyithassák az ajtót belülről.

Emellett a mikrovezérlő szoftveres tervezésénél egy automatikus visszazárási (auto-relock) mechanizmus implementációja is szükséges. Ez garantálja, hogy egy sikeres nyitási parancs után (legyen az manuális, BLE alapú, vagy a fizikai gomb megnyomása), a zár egy előre meghatározott időablak (például 5 másodperc) letelte után automatikusan visszaálljon zárt állapotba. Ez a megoldás kivédi az ajtó véletlen nyitva felejtését.

2.3. Az Android mobilalkalmazás tervezése

A mobilalkalmazás a rendszer felhasználói felületét biztosítja. Főbb feladatai közé tartozik a zárok állapotának megjelenítése és a BLE proximity alapú automatizáció kezelése.

2.3.1. Felhasználói felület és funkcionális követelmények

Az interfész kialakításánál az átláthatóság volt az egyik legfontosabb tervezési szempont. Az alkalmazás fő nézetei (Home, Devices, Logs, Settings) áttekinthetően biztosítják a rendszer állapotának monitorozását és a rendelkezésre álló funkciók elérését. A főbb funkciók közé tartozik:

- **Állapotkövetés:** Valós idejű visszajelzés a zár állapotáról (nyitva/zárva) a Firebase Realtime Database-en keresztül.
- **Manuális vezérlés:** A zárok közvetlen, internetalapú nyitása és zárása.
- **Konfiguráció:** A felhasználói preferenciák, mint például az automatikus nyitás küszöbértékének (RSSI) testreszabása.

2.3.2. Hozzáférés-kezelés és jogosultsági modell

Támogatott a többfelhasználós hozzáférés, ahol két alapvető szerepkör különíthető el:

- **Tulajdonos (Owner):** Teljeskörű jogosultsággal rendelkezik a zár felett, képes új felhasználókat hozzáadni, hozzáférést visszavonni és a zár beállításait módosítani.
- **Vendég (Guest):** Korlátozott hozzáféréssel rendelkezik, elsősorban a zár nyitására és zárására jogosult, az adminisztrációs felületekhez nincs hozzáférése.

Funkcióként implementálásra került az időalapú hozzáférés támogatása is. Ez lehetővé teszi, hogy a vendég jogosultsága egy előre meghatározott idő után automatikusan lejárjon.

2.3.3. Hibrid helymeghatározás: Geofencing és állapotgép

Az automatikus nyitás megvalósítása egy kétszintű állapotgép (State Machine) tervezésén alapul, amelynek célja a telefon akkumulátor kímélése:

1. **Külső zóna (Geofencing):** A telefon GPS helymeghatározást használva figyeli a felhasználó távolságát az otthontól. Amíg a felhasználó távol van (rádiuszon kívül), a rendszer inaktív, nem fogyaszt energiát.

2. **Belső zóna (közelség):** Amíg a felhasználó a geofence zónán belül tartózkodik, a rendszer élesített (aktív) állapotban van: a telefon sugározza a BLE-azonosítóját (beacon), az okosár pedig figyel (szkenneli) a környezetét, és a vett jelerősség alapján dönt a nyitásról.

A két réteg viszonya hierarchikus: a BLE a proximitás alapját adja, a geofencing pedig energiatakarékos kiegészítőként szolgál. A rendszer alapértelmezetten élesített állapotban van, hogy a felhasználó az ajtóhoz érve azonnal nyitni tudjon. A GPS-alapú geofencing feladata a rendszer szüneteltetése. Amikor a telefon helyzete megbízhatóan és tartósan a zónán kívül van a telefon leállítja a beacon sugárzását, és az adatbázison keresztül jelzi az ESP-nek is. Az ESP válaszul felfüggeszti a BLE-szkennelést. A zónán kívül így mindkét eszköz energiát takaríthat meg.

2.4. Kommunikációs modell és felhőalapú adattárolás

Két, jellegében eltérő kommunikációs csatornára épül a rendszer. Ahogyan az architektúra áttekintésénél már említésre került, a távoli vezérlés és az állapotszinkronizáció a felhőadatbázison keresztül valósul meg, a proximity alapú nyitáshoz pedig közvetlen BLE kapcsolat szükséges. Ez az alfejezet az előbbi, felhőalapú csatorna tervezési döntéseit tárgyalja. A felhasznált adatbázistechnológia tulajdonságait a 3. fejezet, a konkrét adatszerkezetet és a hozzáférési szabályokat pedig a 4. fejezet részletezi.

2.4.1. Aszinkron, adatbázis alapú vezérlés

A rendszerarchitektúra kialakítása során cél volt a közvetlen hálózati összeköttetés (P2P) elkerülése a kliensalkalmazás és a hardver között. A közvetítőrétteg kiválasztása így egy felhőadatbázisra esett. Ezzel a tervezési döntéssel megszűnt az a korlátozás, hogy az eszközöknek azonos hálózaton kell tartózkodniuk. A közvetett, aszinkron kommunikáció előnye a robusztusság is. Hálózati kimaradás esetén a kiküldött parancsok nem vesznek el. Az adatbázis eltárolja az új állapotot, amit a hardver a kapcsolat helyreállása után azonnal lekérdez és végrehajt.

2.4.2. A parancs és az állapot szétválasztása

A megbízható visszajelzés érdekében a tervben elkülönül egymástól a kívánt művelet (command) és a zár állapota (status). A mobilalkalmazás jelzi a parancsot (OPEN/CLOSE) az adatbázisnak, a zár állapotát (UNLOCKED/LOCKED) viszont az ESP írja vissza, miután végrehajtotta a parancsot. Az alkalmazás erre az állapotváltozására iratkozik fel, hogy a valós helyzetet tükrözze és ne a feltételezett eredményt. Ez továbbá megakadályozza, hogy a felület egy sikertelen vagy késleltetett művelet esetén téves állapotot mutasson.

2.4.3. Adatmodell és szerveroldali jogosultságkezelés

Az adatmodell tervezésénél fontos volt a felelősségi körök elkülönítése. A rendszernek az alábbi adatokat kell tárolnia, egymástól elkülönítve:

- felhasználók azonosító adatai

- záruk állapota és konfigurációja
- felhasználónkénti hozzáférési jogosultságok
- nyitási események naplója

Biztonsági szempontból a jogosultság-ellenőrzés nem a kliens feladata – a kliens ugyanis megkerülhető. Ezért az ellenőrzés szerveroldalon történik, az adatbázis hozzáférési szabályain keresztül.

Így egy felhasználó csak azokat a zárat vezérelheti, és csak azok naplóit olvashatja, amelyekhez ténylegesen joga van.

2.5. Közelségérzékelés (proximity sensing) és az RSSI-szűrés tervezési kérdései

Az egyik fő tervezési kérdés az volt, hogy miként dönthető el megbízhatóan, hogy a jogosult telefon közelít a zárhoz. Mivel a BLE nem alkalmas távolságmérésre, a rendelkezésre álló mérőszám csak a vett jelerősség (RSSI). Ennek a tulajdonságait és korlátait a 3. fejezet fejti ki. Ez az alfejezet részletezi, milyen feldolgozási és döntési logikával lehet a zajos nyers jelből stabil nyitási döntést hozni.

2.5.1. A nyers jel kezelésének tervezési követelményei

A nyers RSSI állandó távolságnál is jelentősen ingadozik. Emiatt közvetlenül nem viszonyítható jól egy meghatározott küszöbértékhez (threshold). Az apró ingadozások a küszöb közelében folyamatos, oda-vissza kapcsolgathoz vezethetnek. Ebből a tervezés szintjén két követelmény adódik. Egyrészt a nyers RSSI mintákat a döntés előtt korrigálni kell, hogy a kiugró értékek ne befolyásolják a döntést. Másrészt a döntési logikának is védettnek kell lennie a küszöb körüli billegéssel szemben. Ezt az biztosítja, hogy a nyitás csak tartós, küszöb feletti közelség esetén aktiválódik. Automatikus visszazárás után pedig a zár nem nyithat újra azonnal, csak miután a felhasználó ténylegesen eltávolodott.

2.5.2. A szűrési eljárás mint nyitott kérdés

A jelsimításhoz több, eltérő bonyolultságú szűrési eljárás is szóba jöhet. A dolgozat egyik célkitűzése: tapasztalaton alapuló vizsgálat, hogy melyik eljárás vezet a legjobb egyensúlyhoz a zajcsökkentés és a reakcióidő között. Ezért a tervezés szintjén a szűrés kötelező feldolgozási lépésként lett rögzítve. A konkrét eljárás megválasztását viszont nyitva hagyva. Az éles rendszerben alkalmazott megoldást a 4. fejezet, az eljárások összehasonlító értékelését pedig az 5. fejezet tárgyalja.

2.5.3. A küszöbérték testreszabhatósága

Az adott RSSI értékhez tartozó távolság eszközönként és környezetenként eltér. A nyitási küszöb emiatt nem lehet egy kódba égetett állandó. Ezért a tervben a küszöbérték a mobilalkal-

mazásból egy testreszabható paraméter. Ennek köszönhetően a rendszer a telepítés helyéhez igazítható.

2.5.4. Tervezési korlát: az irány megkülönböztethetlensége

Fontos, a tervezés szintjén tudatosított korlát: az RSSI önmagában nem ad információt a megközelítési irányról, tehát nem állapítható meg, hogy a felhasználó az ajtó belső vagy külső oldalán tartózkodik. Ez azt jelenti, hogy a proximity nyitás elvileg belülről, az ajtó előtt elhaladva is aktiválódhat. Ennek a korlátnak a kezelése (például irányérzékeny vagy beltéri/kültéri jeladók kombinálása) jelenleg túlmutat a dolgozat keretein. A rendszer ismert biztonsági korlátai között viszont fontos megemlíteni.

3. fejezet

Felhasznált technológiák

A fejezetben bemutatásra kerülnek a rendszer megvalósításához felhasznált szoftveres és hardveres technológiák. Míg a 2. fejezet a választott irányok és a felépítés döntéseit indokolta, a következőkben az alkalmazott eszközök, azok jellemzőik és működési elvük szerint kerülnek ismertetésre.

3.1. Hardveres komponensek és perifériák

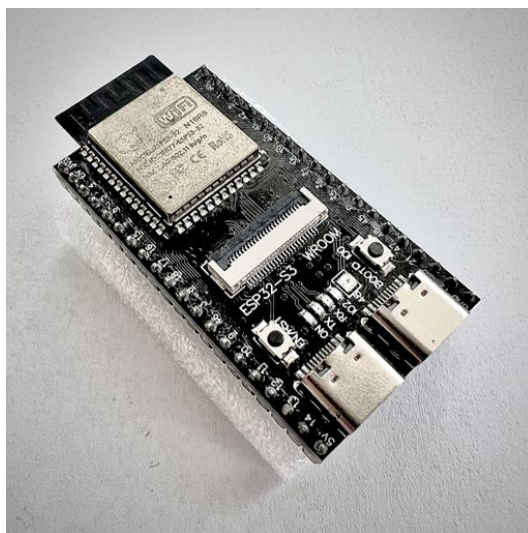
A zár vezérléséért egy ESP32-S3-WROOM-1 mikrovezérlő felel, de ezt a folyamatot több elektronikai modul együttműködése teszi lehetővé. A komponensek megválasztásánál fontos szempont volt a költséghatékonyság, a megbízhatóság és az egyszerű integrálhatóság.

3.1.1. Az ESP32-S3-WROOM vezérlőegység

A rendszer központi vezérlőegysége ESP32-S3 WROOM-1 N16R8 modul, amelyet az Espressif Systems fejleszt [1]. Gyakran alkalmazott IoT-rendszerekben a beépített vezeték nélküli kapcsolatok és az alacsony energiafogyasztás miatt.

A modul jellemzői:

- **Processzor:** kétmagos Xtensa® 32 bites LX7, akár 240 MHz órajellel;
- **Vezeték nélküli kapcsolat:** 2,4 GHz-es Wi-Fi (802.11 b/g/n) és Bluetooth 5 (LE);
- **Memória:** 512 KB SRAM, 16 MB flash és 8 MB PSRAM (N16R8 variáns);
- **Ki- és bemenetek:** legfeljebb 36 kivezetett GPIO, ADC, I²C, SPI, UART, valamint LED PWM perifériák;
- **Tápfeszültség:** 3,3 V logikai szint.



2. ábra. ESP32-S3 WROOM-1 N16R8 modul

Az ESP32-S3 a következő funkciókért felel:

- BLE-jelek figyelése
- RSSI feldolgozása
- kétirányú kommunikáció a Firebase adatbázissal
- a zárszerkezet relén keresztüli vezérlése

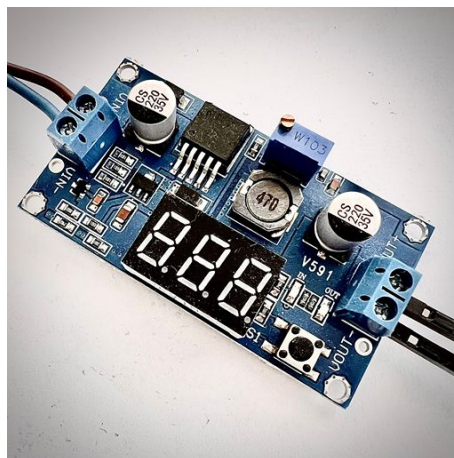
A beépített Wi-Fi és Bluetooth moduloknak köszönhetően mindezeket önállóan ellátja, külön központi számítógép (például Raspberry Pi) nélkül.

3.1.2. Tápellátás és feszültségszabályozás

A rendszer komponensei eltérő feszültség szinteken működnek, a tápellátás többlépcsős. A rendszer egy 2 A terhelhetőségű (összesen 24 W) adatterről üzemel, amely a 100-240 V-os hálózati feszültséget 12 V-os egyenfeszültségre alakítja. Az adapter az áramkörhöz egy 5,5/2,1 mm-es DC-044 típusú, panelre szerelhető csatlakozóaljzaton keresztül kapcsolódik.



(a) DC-044 tápcsatlakozó-aljzat



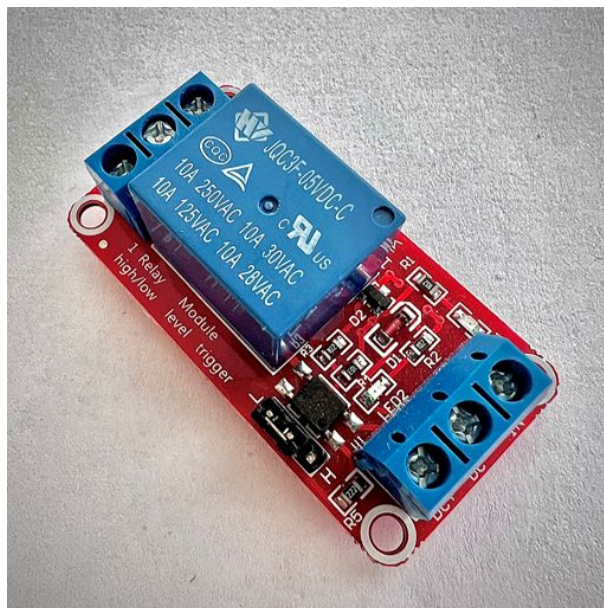
(b) LM2596 step-down feszültségszabályzó

3. ábra. A tápellátás komponensei

A 12 V-os tápfeszültség táplálja a zárszerkezet szolenoidját és a relé modult. A mikrovezérlő ezzel szemben 5 V-os bemeneti feszültséget igényel, ezért a 12 V feszültség csökkentését egy LM2596 típusú kapcsolóüzemű (step-down) konverter [2] biztosítja. Az LM2596 állítható kimenetű és rendelkezik beépített feszültségkijelzővel. Ez beüzemelés során megkönnyíti a kimeneti feszültség pontos beállítását.

3.1.3. Relé modul optocsatolóval

A mikrovezérlő 3,3 V-os logikai kimenete nem képes közvetlenül kapcsolni a zárszerkezet 12 V-os áramkörét, ezért a kapcsolást egy egycsatornás relé modul (HW-803, JQC3F-05VDC-C) végzi. A modul optocsatolóval van felszerelve, amely a mikrovezérlőt galvanikusan elválasztja a relé meghajtó oldalától [3]. Így a kapcsoláskor keletkező zavarok és feszültségtüskék nem jutnak vissza a mikrovezérlőre.



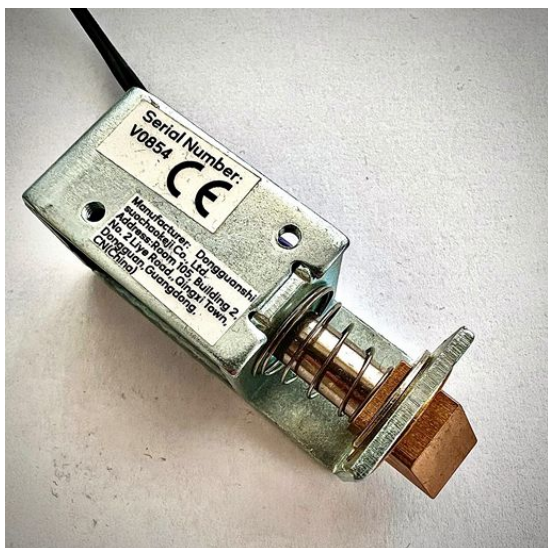
4. ábra. Optocsatolós egycsatornás relé modul (HW-803, JQC3F-05VDC-C)

A modul magas és alacsony szintű kapcsolást (high/low level trigger) támogat. A mikrovezérlő LOW logikai szintje zárja a relét. Ez feszültség alá helyezi a szolenoidot és kinyitja a zárat. A relé így kapcsolóként működik a mikrovezérlő és a zárszerkezet között.

3.1.4. Szolenoidos zár és a védődióda

A tényleges zárszerkezet egy 12 V-os, billenőnyelvű szolenoidos zár (Tilt Tongue Solenoid Lock). Működési elve, hogy feszültség alá helyezve a szolenoid behúzza a záró nyelvet, ezzel kioldva a zárat. Feszültségmentes állapotban egy rugó visszaállítja azt zárt helyzetbe. Ez a megoldás eleve biztonságos (fail-secure). Áramkimaradás esetén a zár automatikusan zárt állapotba kerül. A szolenoid csak a nyitás időtartamára kap feszültséget, így ez a megoldás energiatakarékos, és megelőzi a tekercs túlmelegedését.

A szolenoid lényegében egy tekercs, vagyis induktív terhelés. Amint lekapcsolódik róla a feszültség, a mágneses térben tárolt energia nem tud azonnal eltűnni. Egy rövid, nagy fe-



(a) Billenőnyelvű szolenoidos zár



(b) 1N4007 flyback védődióda

5. ábra. A zárszerkezet és a túlfeszültség-védelem

szülségtüske (inductive kickback) formájában jelenik meg [3]. Ezt a tüskét vezeti le a tekercssel párhuzamosan bekötött 1N4007 flyback védődióda (1 A névleges áram, 1000 V záróirányú feszültség) [4]. Normál működés közben a dióda zár, a kikapcsoláskor keletkező fordított tüskét viszont rövidre zárja, amivel megóvjá a relé érintkezőit és a többi alkatrészt.

3.2. Bluetooth Low Energy és RSSI

3.2.1. A Bluetooth Low Energy működési elve

A Bluetooth Low Energy (BLE) a Bluetooth szabvány alacsony energiafogyasztású változata. A BLE kifejezetten a kis adatmennyiségű kommunikációt igénylő eszközökhöz lett kifejlesztve. 2,4 GHz-es ISM-sávban működik [5]. BLE esetén két lényegi üzemmódot különböztetünk meg:

- **Hirdetési (advertising) mód:** kapcsolat létesítése nélkül a készülék periodikusan kis adatcsomagokat sugároz, amelyeket a környezetében lévő szkennerek (scanner) eszközök foglalkoznak (receive)
- **Kapcsolódási (connected via GATT) mód:** két eszköz tartós kapcsolatot épít ki, és szolgáltatások, illetve karakterisztikák (characteristics) mentén kétirányú adatcserét folytat.

	Hirdetési (advertising)	Kapcsolódási (GATT)
Kapcsolat	nincs, egyirányú szórás	tartós, kétirányú
Adatmennyiség	kis csomag, periodikus	szolgáltatások/karakterisztikák
Energia	alacsony	magasabb
Szerepe	proximity nyitás	üzembe helyezés (konfiguráció)

3.1. táblázat. A BLE két üzemmódjának összehasonlítása

A rendszer mindkét üzemmódot kihasználja: a mindennapi, közelség alapú nyitáshoz a telefon hirdetési módban sugározza az azonosítóját, az okosár pedig figyelőként veszi azt,

míg az okosvár üzembe helyezésekor a GATT-alapú kapcsolat biztosítja a konfigurációs adatok biztonságos átvitelét.

3.2.2. Az RSSI mint közelségi mutató

A vett jelerősség (Received Signal Strength Indicator, RSSI) azt fejezi ki, mekkora teljesítménnyel érkezik a rádiójel a vevőhöz, mértékegysége a decibel- milliwatt (dBm). Az érték jellemzően negatív: a nullához közelebbi (például -40 dBm) érték erős, közeli jelet, a kisebb (például -90 dBm) érték gyenge, távoli jelet jelez. Mivel a jel teljesítménye a távolsággal csökken, az RSSI közvetett, olcsó és külön hardvert nem igénylő módszert ad a két eszköz közötti távolság becslésére, ezért gyakori a közelség alapú alkalmazásokban.

Az RSSI azonban nem alkalmas a pontos távolságmérésre. Az érték erősen ingadozik a többutas terjedés (multipath), az emberi test okozta jelárnyékolás, valamint a környezeti interferencia miatt [6], így állandó távolság mellett is jelentősen eltérhetnek az egymást követő mérések. Éppen ezért a nyers RSSI-értékek közvetlenül nem használhatók megbízható nyitási döntéshez, hanem szűrésre van szükség, amelynek vizsgálata a dolgozat központi témáját képezi.

3.2.3. A NimBLE könyvtár

Az ESP32 BLE-funkcióit a NimBLE könyvtár valósítja meg. A NimBLE az alapértelmezett Bluetooth veremnél lényegesen kisebb igényű implementáció memória és tárhely szempontjából. Előnyösebb a beágyazott környezetben, mivel több erőforrást hagy az alkalmazás többi feladatának [7]. A könyvtár visszahívásos (callback) alapú felülete kényelmes hozzáférést biztosít a hirdetett adatokhoz és a mért jelerősséghez, ami a rendszer "proximity" logikájának alapját adja.

3.3. A Firebase platform

A rendszer szerveroldali részét a Google Firebase platformja adja. Ez egy felhőalapú háttérrendszer (Backend as a Service, BaaS), amelyhez nem kell saját szervert üzemeltetni. A platform biztosítja a hitelesítést, az adattárolást és a valós idejű szinkronizációt. A projekt két szolgáltatását használja: a Firebase Authentication a felhasználók azonosítását, a Firebase Realtime Database pedig az adatok tárolását és a komponensek közötti továbbítását végzi.

3.3.1. Firebase Authentication

A Firebase Authentication a felhasználók azonosítását végzi, és többféle bejelentkezési módot támogat [8]. A rendszer az e-mail-cím és jelszó alapú változatot használja. A regisztráció és a bejelentkezés a szolgáltatás felületén keresztül történik, így a jelszavak tárolásával és titkosításával nem kell külön foglalkozni. Minden bejelentkezett felhasználó egy egyedi azonosítót (UID) kap. Ez köti össze a felhasználói profilt a zárhoz rendelt jogosultságokkal.

3.3.2. Firebase Realtime Database

A Firebase Realtime Database egy felhőben tárolt NoSQL adatbázis, amely az adatokat egyetlen, hierarchikus JSON-fa szerkezetben tárolja [9]. Az adatbázis legfontosabb tulajdonsága a valós idejű szinkronizáció. A kliens nem egyszeri lekérdezésekkel kéri le az adatokat, hanem figyelőket (listener) regisztrál egy adatcsomópontra. Az adatok változásakor azonnal, automatikusan értesül róla. Emiatt a zár állapotának változása megjelenik a mobilalkalmazásban késleltetés nélkül és a telefonról küldött parancs azonnal eljut a zárhoz.

A rendszerben az adatbázis tölti be a központi közvetítő réteg szerepét. Mivel a mikrovezérlő és a mobilalkalmazás nem közvetlenül, hanem ezen az adatbázison keresztül kommunikál, a két eszköznek nem kell azonos hálózaton tartózkodnia, és a zár bárhol, valós időben vezérelhető. Az adatbázis ezen felül tárolja a felhasználói profilokat, a hozzáférési jogosultságokat és a nyitási események naplóját is. Az adatbázis pontos szerkezetét és a hozzáférési szabályait a 4. fejezet részletezi.

3.4. Az Android alkalmazás technológiái

A rendszer felhasználói felületét egy közvetlenül Androidra fejlesztett alkalmazás valósítja meg. Erre azért volt szükség, mert a "proximity" alapú automatikus nyitás megköveteli a Bluetooth Low Energy és a helymeghatározás alacsony szintű elérését.

3.4.1. Kotlin és az Android keretrendszer

Az alkalmazás Kotlin nyelven készült, amely 2019 óta a Google által elsődlegesen ajánlott nyelv az Android-fejlesztéshez [10].

Az alkalmazás a Google által ajánlott architektúrát követi [11]: a képernyőket Activity és Fragment elemek építik fel, az állapotkezelést pedig az AndroidX komponensei (ViewModel, LiveData) végzik. Ez a felelősségek szétválasztásával átláthatóbbá és könnyebben karbantarthatóvá teszi a kódot.

3.4.2. Felhasznált könyvtárak és fejlesztői eszközök

A fejlesztés az Android Studio környezetben, a Gradle build-rendszerrel történt. Az alkalmazás több külső könyvtárat is használ:

- **Firebase SDK:** a hitelesítés (`firebase-auth`) és az adatbázis-kapcsolat (`firebase-database`) kezelésére;
- **Google Play Services Location:** a geofencinghez szükséges helymeghatározásra, a `FusedLocationProvider` interfészen keresztül;
- **Material Components és RecyclerView:** a felület egységes megjelenéséhez és a listák hatékony kezeléséhez;
- **Android Bluetooth LE API:** a beacon-jel sugárzásához (`advertising`) és az üzembe helyezéskor a zárral való kapcsolódáshoz.

A vezeték nélküli kommunikáció kliensoldali része így teljes egészében az Android beépített Bluetooth- és helymeghatározó szolgáltatásaira épül.

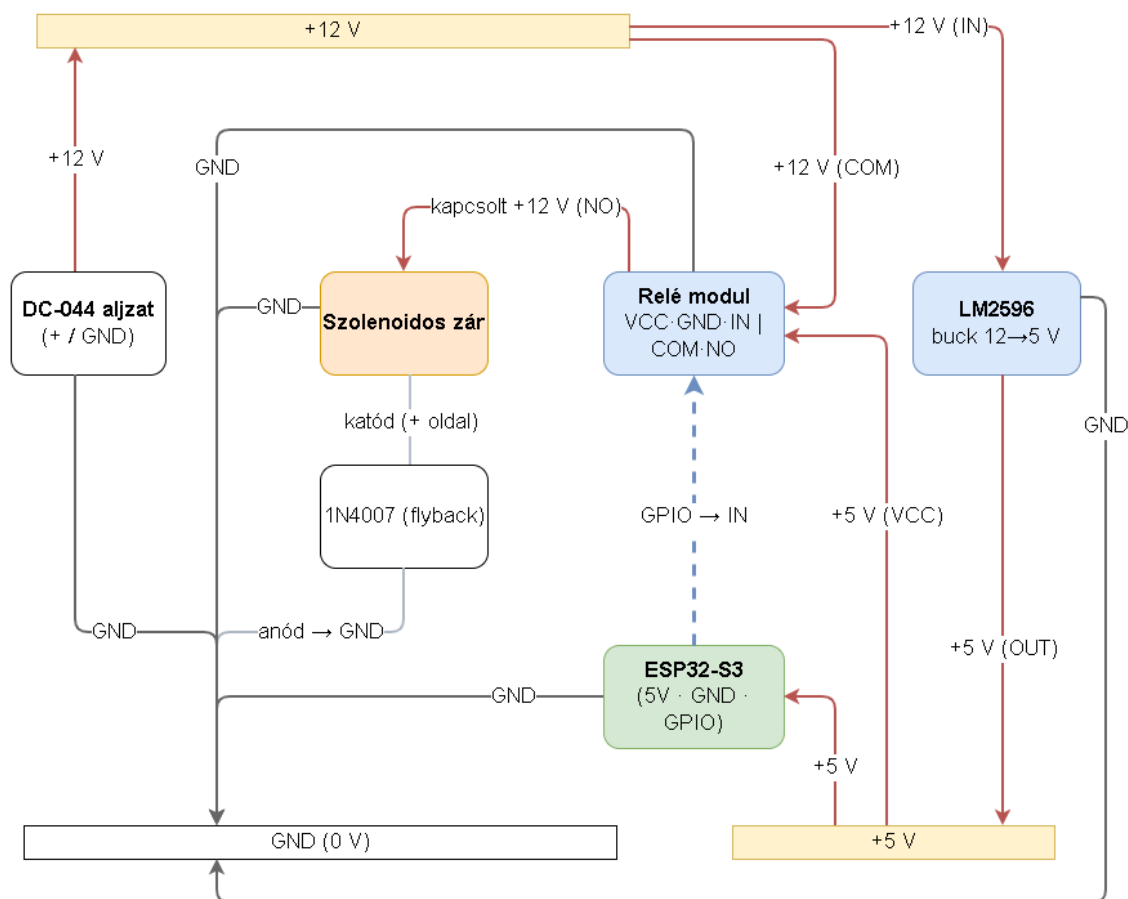
4. fejezet

Megvalósítás

A megvalósítási szakasz a 2. fejezetben lefektetett architektúra és a kiválasztott technológiák alapján történt. A cél egy működőképes prototípus létrehozása volt, amelyben a három komponens (ESP32 firmware, Firebase RTDB és Android alkalmazás) összehangoltan valósítja meg az okosvár funkcióit. A következő alfejezetek az egyes komponensek megvalósítását és a legfontosabb kódrészleteket mutatják be. A leírás logikája az adatok áramlását követi. A teljes forráskód terjedelmi okokból nem szerepel a dolgozatban. Az egyes alfejezetek csak a működés megértéséhez szükséges, releváns részleteket emelik ki.

4.1. A hardver összeállítása és bekötése

A 3. fejezetben bemutatott komponensekből összeállított prototípus bekötését a 6. ábra szemlélteti. A tápellátás három különböző feszültség szintet kezel: a DC-044 aljzaton beérkező 12 V közvetlenül a relét és a szolenoidot táplálja, az LM2596 ebből állítja elő az ESP32 számára szükséges 5 V-ot, a közös GND-sín pedig minden modult összeköt. Az ESP32 egy GPIO-kimenete vezérli a relét, amely a 12 V-os ágot kapcsolja a szolenoidra. A szolenoiddal párhuzamosan kötött 1N4007 dióda védi az áramkört a kikapcsoláskor keletkező feszültségtüskéktől.



6. ábra. Az okosár rendszer bekötési rajza

A perifériák ESP32-S3 mikrovezérlőhöz rendelt lábkiosztását a 4.1. táblázat foglalja össze.

Periféria	ESP32-S3 GPIO	Mód
Relé (zárvezérlés)	GPIO 13	kimenet (LOW = nyit)
Állapotjelző LED	GPIO 2	kimenet
Felülbíró nyomógomb	GPIO 0	bemenet (pull-up)

4.1. táblázat. A perifériák GPIO-kiosztása

4.2. Az ESP32 firmware

Az ESP32 firmware C++ nyelven, az Arduino keretrendszer alapján készült. A Bluetooth Low Energy műveletekhez a NimBLE könyvtárat, az adatbázissal való kommunikációhoz a Firebase_ESP_Client könyvtárat, a hálózati beállításhoz pedig a WiFiManager könyvtárat használja.

4.2.1. Inicializálás és az eszközazonosító generálása

A bekapcsolást követően a firmware először a Wi-Fi kapcsolatot építi fel a WiFiManager segítségével. Ez a megoldás lehetővé teszi, hogy a hálózati hozzáférési adatokat ne kelljen a forráskódba égetni. Sikertelen csatlakozás (korábban mentett hálózathoz) vagy hiányzó hálózati

beállítások esetén az eszköz egy ideiglenes konfigurációs hozzáférési pontot nyit. Ezen a felületen keresztül a felhasználó könnyedén megadhatja a szükséges Wi-Fi adatokat.

A sikeres csatlakozás után az eszköz a saját MAC-címéből egyedi azonosítót képez. Ezzel biztosítható, hogy minden fizikai zár külön konfiguráció nélkül, egyértelmű és ütközésmentes azonosítóval szerepeljen az adatbázisban.

4.1. Listing. Az eszközazonosító képzése a MAC-címből

```
1 String getMacFormatted() {
2     String mac = WiFi.macAddress();
3     mac.replace(":", "");
4     mac.toUpperCase();
5     return "LOCK_" + mac;    // pl. LOCK_0CDC7E614160
6 }
```

Ezt követően a firmware bejelentkezik a Firebase szolgáltatásba egy dedikált eszközfiókkal, majd lekérdezi az adatbázisból az *activated* mező értékét. Ez dönti el, hogy az eszköz az üzembe helyezési (aktivációs) vagy a normál működési módban indul el.

4.2.2. Üzemmodok: aktivációs és normál működés

A firmware két, élesen elkülönülő üzemmódot valósít meg. Egy még nem aktivált zár az *aktivációs módba* lép: ekkor az ESP32 BLE GATT-szerverként hirdeti magát, és megvárja, amíg a mobilalkalmazás csatlakozik hozzá, hogy átvegye a tulajdonos azonosítóját és a hozzá tartozó beacon-azonosítót. Egy már aktivált zár ezzel szemben a *normál működési módban* indul: feliratkozik a saját adatbázis- csomópontjának változásaira, és — a közelségi funkció állapotától függően — megkezdja a környezet BLE alapú szkennelését. A két mód közötti döntést a korábban lekérdezett *activated* mező vezérli.

4.2.3. BLE szkennelés és a jogosult beaconök azonosítása

Normál működés során az ESP32 a környezetében sugárzó BLE-eszközöket figyeli. A szkennelés nem fut folyamatosan: csak akkor aktív, ha a közelségi funkció be van kapcsolva (*bleProximityEnabled*), amelyet a mobilalkalmazás a hibrid logika alapján vezérel. Ezzel elkerülhető a fölösleges energifogyasztás, amikor egyetlen jogosult eszköz sincs a közelben.

Minden észlelt eszköz esetén a firmware kétszintű ellenőrzést végez. Először megvizsgálja, hogy a hirdetés tartalmazza-e a rendszer egyedi szolgáltatás- azonosítóját (0xABCD), majd a hirdetett adatban szereplő beacon-azonosítót összeveti a zárhoz rendelt, jogosult azonosítók listájával. Csak teljes egyezés esetén dolgozza fel a vett jelerősséget, így a zár kizárólag a regisztrált eszközök jelére reagál.

4.2. Listing. A jogosult beaconök szűrése és az RSSI kiolvasása

```
1 void onResult(const NimBLEAdvertisedDevice* dev) override {
2     if (!dev->haveServiceData()) return;
3     for (int s = 0; s < dev->getServiceDataCount(); s++) {
4         // A rendszer saját szolgáltatás-azonosítója: 0xABCD
5         if (dev->getServiceDataUUID(s).equals(NimBLEUUID((uint16_t)0xABCD))) {
```

```

6      String found = serviceDataToHex(dev->getServiceData(s));
7      for (String u : authorizedBeacons) {
8          if (found.equalsIgnoreCase(u)) {
9              int rssi = dev->getRSSI();    // a nyers jelerosseg (dBm)
10             updateRssiWindow(rssi);      // tovabbitas a szuronek
11             break;
12         }
13     }
14 }
15 }
16 }

```

4.2.4. Az RSSI-szűrés megvalósítása

A vett jelerősség nyers értékei a többutas terjedés, a testtakarás és a környezeti interferencia miatt erősen zajosak, így közvetlenül nem alkalmasak megbízható nyitási döntés meghozatalára. Ezért az éles rendszer a nyers mintákat egy exponenciális mozgóátlag (Exponential Moving Average, EMA) szűrőn vezeti át a döntés előtt. A szűrő minden új mintát egy súlyozott átlagba épít be, ahol az α simítási tényező határozza meg a zajcsökkentés és a reakcióidő közötti egyensúlyt:

$$EMA_t = \alpha \cdot RSSI_t + (1 - \alpha) \cdot EMA_{t-1} \quad (4.1)$$

A megvalósításban $\alpha = 0,35$, amely a tapasztalatok alapján kellően gyors reakciót biztosít a közeledésre, miközben elnyomja a pillanatnyi kiugró értékeket. A szűrt érték egy rögzített időablakonként frissül, az adott ablakban mért legerősebb mintából.

4.3. Listing. Az EMA-szűrő megvalósítása az éles firmware-ben

```

1  const float EMA_ALPHA = 0.35f;
2  float emaMaster = -100.0f;
3
4  emaMaster = EMA_ALPHA * (float)bestMasterRssiThisWindow
5             + (1.0f - EMA_ALPHA) * emaMaster;

```

A nyitási döntés a szűrt értéken alapul: ha az meghaladja a beállított küszöböt, a zár zárt állapotban van, és nem áll fenn térbeli visszazárási tiltás, a firmware nyit. A térbeli visszazárási védelem (spatialCooldown) megakadályozza, hogy a zár az automatikus visszazárás után azonnal újra kinyisson, amíg a felhasználó ténylegesen el nem távolodott.

4.4. Listing. A nyitási döntés a szűrt jelerősség alapján

```

1  if (emaMaster >= bleRssiThreshold && isLocked && !spatialCooldownActive) {
2      unlockDoor("BLE");
3      spatialCooldownActive = true;    // ujranyitas tiltasa, amig el nem
4                                       tavolodik
5  }

```

Az éles rendszerben tehát kizárólag az EMA-szűrő fut. A dolgozat egyik fő célja azonban annak vizsgálata, hogy más szűrési eljárások jobb eredményt adnának-e. Ezt a kérdést (az éles rendszertől függetlenül, rögzített mérési adatokon) az 5. fejezet vizsgálja.

4.2.5. Zárvezérlés és automatikus visszazárás

A zárszerkezetet a firmware egy digitális kimeneten keresztül, a relé vezérlésével működteti. A nyitáskor az eszköz feszültség alá helyezi a szolenoidot, és eltárolja a nyitás módját (manuális, BLE alapú vagy a fizikai gomb általi), amelyet később a naplózás használ fel. A 2. fejezetben ismertetett biztonsági követelménynek megfelelően a firmware automatikus visszazárást valósít meg: egy sikeres nyitás után rögzített idő (a megvalósításban 5 másodperc) elteltével a zár magától visszaáll zárt állapotba.

4.5. Listing. Az automatikus visszazárás a fő ciklusban

```
1 // AUTO RE-LOCK: a nyitás után 5 másodperccel újra zár
2 if (!isLocked && (now - unlockTime >= AUTO_RELOCK_MS)) {
3     lockDoor();
4 }
```

4.2.6. Firebase-stream és a parancsfeldolgozás

A zár és a mobilalkalmazás közötti kommunikáció a Firebase adatbázison keresztül valósul meg. A firmware egyetlen adatfolyamot (stream) nyit a zár saját adatcsomópontjára, így a parancsok, a küszöbértékek és a jogosult beacon-ok változásairól azonnal értesül. Fontos megvalósítási szempont volt, hogy az adatfolyam visszahívásából (callback) közvetlenül nem indítható újabb adatbázis-művelet, mert az megszakítja a biztonságos kapcsolatot. Ezért a callback csak jelzőket állít be, a tényleges feldolgozás pedig a fő ciklusban történik.

4.6. Listing. A parancs feldolgozása a fő ciklusban, nem a visszahívásban

```
1 if (fbReady && pendingCmdProcess) {
2     pendingCmdProcess = false;
3     applyCommand(pendingCmd); // "OPEN" -> nyitas, "CLOSE" -> zaras
4 }
```

4.3. A Firebase adatbázis

Az adatbázis a rendszer központi közvetítő rétege: ezen keresztül kommunikál az ESP32-S3 és a mobilalkalmazás. A 2. fejezetben ismertetett tervezési elveket (aszinkron vezérlés, a parancs és az állapot szétválasztása, szerveroldali jogosultságkezelés) az alábbi konkrét adatszerkezet valósítja meg.

4.3.1. Az adatfa szerkezete

Az adatok négy fő ág alá rendeződnek. A locks ág tárolja az egyes zárok állapotát és konfigurációját, a permissions a felhasználónkénti jogosultságokat, a users a profiladatokat, a logs pedig a nyitási események naplóját.

4.7. Listing. A Firebase Realtime Database adatfájának vázlata

```
1 {
2     "users": {
```

```

3     "<uid>": { "email": "...", "displayName": "...", "uid": "<uid>" }
4 },
5 "locks": {
6     "<lockId>": {
7         "status":          "LOCKED",    // a zár tenyleges állapot (ESP
8             irja)
9         "command":         "NONE",      // OPEN / CLOSE / NONE (app írja)
10        "lastUnlockMethod": "MANUAL",    // MANUAL / BLE / MANUAL_INTERNAL
11        "activated":        true,
12        "name":              "Front Door",
13        "macAddress":        "B8:F8:62:E0:BC:BC",
14        "rssiThreshold":     -60,
15        "bleProximityEnabled": true,
16        "authorizedBeacons": { "<uid>": "<beaconId>" },
17        "owner":             "<uid>",
18        "location":          { "lat": 46.53, "lng": 24.57 }
19    }
20 },
21 "permissions": {
22     "<lockId>": {
23         "<uid>": { "role": "owner", "grantedAt": 1780512396789 },
24         "<uid>": { "role": "guest", "expiresAt": 1783157003976 }
25     }
26 },
27 "logs": {
28     "<pushId>": {
29         "lock_id": "<lockId>", "method": "MANUAL", "user_email": "...",
30         "formatted_date": "2026-06-03 20:00:27", "timestamp": 1780506027100
31     }
32 }

```

A parancs és az állapot szétválasztása itt válik kézzelfoghatóvá: az alkalmazás a `command` mezőt írja (OPEN/CLOSE), az ESP32 pedig a végrehajtás után a `status` mezőt frissíti (LOCKED/UNLOCKED). A futás közben módosítható beállítások (a nyitási küszöb `rssiThreshold` és a proximity funkciót kapcsoló `bleProximityEnabled`) szintén a zár csomópontja alatt található; ez utóbbit az alkalmazás hibrid logikája vezérli.

4.3.2. Szerveroldali jogosultságkezelés

A jogosultság-ellenőrzés nem a kliensben, hanem az adatbázis biztonsági szabályaiban (Security Rules) van. Egy felhasználó csak akkor olvashat vagy írhat egy zárat, ha szerepel a `permissions` ágban az adott zárhoz rendelve. Így a kliens megkerülésével sem lehet jogosulatlanul hozzáférni egy zárhoz.

4.8. Listing. A zárokra vonatkozó hozzáférési szabály (kivonat)

```

1 "locks": {
2     "$lockId": {
3         ".read": "root.child('permissions').child($lockId).child(auth.uid).
                exists()",

```

```

4     ".write": "root.child('permissions').child($lockId).child(auth.uid).
        exists()"
5   }
6 }

```

A tulajdonosi (owner) és vendég (guest) szerepkör közötti különbséget, valamint a vendég jogosultság lejáratát (expiresAt) szintén a permissions ág és a hozzá tartozó szabályok kezelik.

4.4. Az Android alkalmazás

Az alkalmazás megvalósításának két kulcseleme a 2. fejezetben tervezett hibrid helymeghatározás és a BLE-beacon sugárzása. A felépítést (Kotlin, AndroidX architektúra) a 3. fejezet ismertette. Ez a szakasz a működés megvalósítását mutatja be.

4.4.1. A hibrid helymeghatározás megvalósítása

A kétszintű logika külső rétegét az Android geofencing szolgáltatása adja. Az alkalmazás egy geofence-zónát regisztrál az otthon köré, és a be-/kilépési eseményekre iratkozik fel. Amikor a telefon tartósan elhagyja a zónát, az alkalmazás leállítja a beacon sugárzását, és az adatbázisban a bleProximityEnabled mezőt false értékre állítja. Ennek hatására az ESP32-S3 is felfüggeszti a szkennelést. Visszatéréskor a folyamat fordítva játszódik le. Így a zónán kívül mindkét eszköz energiát takarít meg.

A belső réteg a tényleges proximity alapú nyitás. A zónán belül az alkalmazás folyamatosan sugározza a zárhoz rendelt beacon azonosítót.

4.4.2. A beacon sugárzása

Az alkalmazás az Android BLE advertising felületén keresztül sugározza az azonosítóját. A hirdetés a rendszer egyedi szolgáltatás-azonosítóját (0xABCD) és a beacon-azonosítót tartalmazza. Az ESP32-S3 pontosan ezt a szolgáltatás azonosítót keresi a szkennelés során, és csak a jogosult beacon-ok jelét dolgozza fel.

4.9. Listing. A beacon-hirdetés összeállítása (Kotlin)

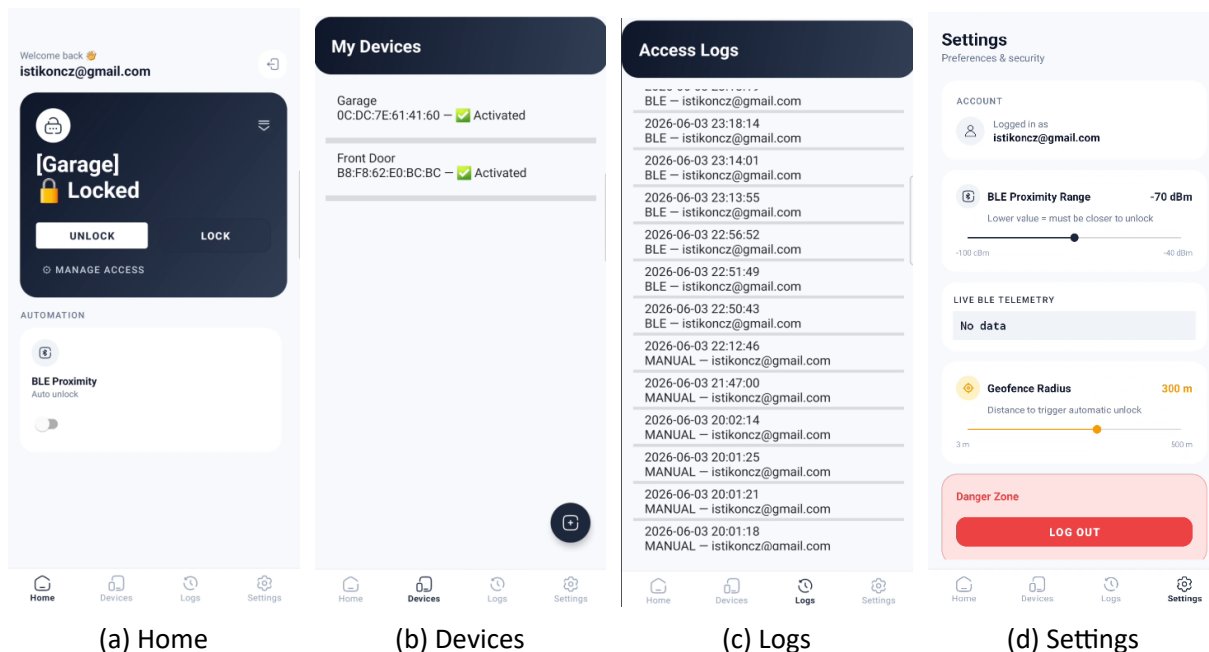
```

1 val data = AdvertiseData.Builder()
2     .addServiceData(
3         ParcelUuid(SERVICE_UUID),    // 0xABCD
4         beaconId.toByteArray()
5     )
6     .build()
7 advertiser.startAdvertising(settings, data, advertiseCallback)

```

4.4.3. A felhasználói felület

Az alkalmazás négy fő nézetből áll. A *Home* a záruk pillanatnyi állapotát és a manuális nyitást/zárást teszi elérhetővé, a *Devices* a záruk hozzáadását és a jogosultságok kezelését, a *Logs* a nyitási események naplóját, a *Settings* pedig a beállításokat, többek között a nyitási küszöb (*rssiThreshold*) testreszabását.



7. ábra. Az Android alkalmazás fő nézetei

5. fejezet

Eredmények

A következő fejezt a megépített rendszer kísérleti és gyakorlati kiértékelést mutatja be. Két, egymást kiegészítő szálon futott a vizsgálat. Az első egy offline teszt sorozat rögzített adatokon, amelynek célja a 2. fejezetben nyitva hagyott kérdésre ad választ. Megtalálni azt az RSSI-szűrés eljárást, amely jó kompromisszumot ad a zajszűrés és a válaszidő között. Ezt követi a második fázis, ahol már az éles rendszer tesztelése történt valós körülmények között. Itt a nyitás gyorsasága, megbízhatósága, a téves nyitások aránya és a hosszú távú stabilitás volt értékelve.

5.1. A mérési módszertan

Az offline összehasonlításhoz egy referencia-adatsort rögzítettem, a telefon ajtó felé közelítést, a nyers RSSI-mintákkal és időbélyegekkkel együtt. Ezen az *azonos* adatsoron futtattam le utólag az összes szűrt és paraméter-beállítást, így az eljárások közvetlenül, azonos bemeneten összehasonlíthatók. Az értékelt mennyiségek:

- **Átlagos küszöb-átlépési idő (*avg_crossing_s*):** mennyivel az ajtóhoz érés előtt (negatív érték) vagy után lépi át a szűrt jel a küszöböt;
- **Szórás (*std_s*):** az átlépési idő ingadozása az ismételt futások között, a megismételhetőség mérőszáma;
- **Téves pozitív (*false_positives*):** hány esetben lépte át a jel a küszöböt indokolatlanul;
- **A szűrt jel simasága (*rms_derivative*):** a szűrt görbe deriváltjának négyzetes átlaga, a maradék zaj mérőszáma.

A nyitás időzítésének méréséhez egy független referenciapontra volt szükség: arra a pillanatra, amikor a felhasználó ténylegesen az ajtóhoz ér. Ezt egy külön, erre a célra készített referencia-eszközzel (egy második ESP32-vel) rögzítettem, amelyen egy gombot az ajtóhoz érkezés pillanatában nyomtam meg. Az eszköz ESP-NOW-n keresztül küldte ezt az időbélyeget a zárnak, így minden kísérletnél össze tudtam vetni a zár tényleges nyitási idejét az ajtóhoz érkezés időpontjával. A mért várakozási idő tehát ehhez a referenciaponthoz képest értendő: a negatív érték azt jelenti, hogy a zár már az ajtóhoz érkezés előtt kinyílt, a pozitív pedig azt, hogy a felhasználónak várnia kellett.

Az éles használati mérés a ténylegesen telepített rendszeren történt, három hordási móddal (kézben, zsebben, táskában tartott telefon) és három RSSI-küszöbön (-60 , -55 , -50 dBm). Külön mérés vizsgálta a téves nyitásokat és a rendszer többnapos stabilitását.

5.2. Az RSSI-szűrők összehasonlítása

A különböző szűrőcsaládok (mozgóátlag nélküli nyers jel, EMA, mediánszűrő, aszimmetrikus EMA és Kálmán-szűrő) viselkedését a 5.1. táblázat foglalja össze a -60 dBm-es küszöbnél. A szűrés hatása itt a legszembevetőbb: a nyers jel átlépési ideje stabilnak tűnik (kis szórás), de 19 téves pozitívet produkál, vagyis a küszöb körüli zaj miatt sokszor indokolatlanul jelez. A szűrők ezzel szemben a téves pozitívak számát nullára vagy közel nullára szorítják.

Szűrő	Beállítás	Átlépés (s)	Szórás (s)	Téves poz.	Simaság
Nyers	–	$-11,98$	$0,66$	19	–
EMA	$\alpha = 0,20$	$-8,20$	$2,11$	0	$1,49$
EMA	$\alpha = 0,35$	$-10,62$	$1,20$	0	$2,43$
EMA	$\alpha = 0,50$	$-11,52$	$0,79$	0	$3,45$
EMA	$\alpha = 0,70$	$-11,71$	$0,67$	3	$5,01$
Medián	$w = 3$	$-11,84$	$0,66$	3	$3,92$
Medián	$w = 5$	$-11,69$	$0,74$	0	$2,56$
Medián	$w = 7$	$-11,24$	$1,18$	0	$2,04$
Medián	$w = 9$	$-11,27$	$1,18$	0	$1,82$
Aszim. EMA	$\uparrow 0,5 \downarrow 0,15$	$-11,54$	$0,78$	1	$2,12$
Aszim. EMA	$\uparrow 0,7 \downarrow 0,15$	$-11,71$	$0,67$	7	$2,56$
Aszim. EMA	$\uparrow 0,7 \downarrow 0,30$	$-11,71$	$0,67$	7	$3,28$
Aszim. EMA	$\uparrow 0,9 \downarrow 0,15$	$-11,78$	$0,71$	7	$2,97$
Kálmán	$Q = 0,1 \ R = 10$	$-5,05$	$0,84$	0	$0,86$
Kálmán	$Q = 1 \ R = 10$	$-10,45$	$1,15$	0	$1,92$
Kálmán	$Q = 5 \ R = 10$	$-11,52$	$0,79$	0	$3,45$
Kálmán	$Q = 10 \ R = 5$	$-11,75$	$0,72$	5	$5,29$

5.1. táblázat. Az RSSI-szűrők összehasonlítása a -60 dBm-es küszöbnél. Az átlépési idő negatív értéke az ajtóhoz érés előtti nyitást jelzi; a simaság a szűrt jel deriváltjának RMS-értéke (kisebb = simább).

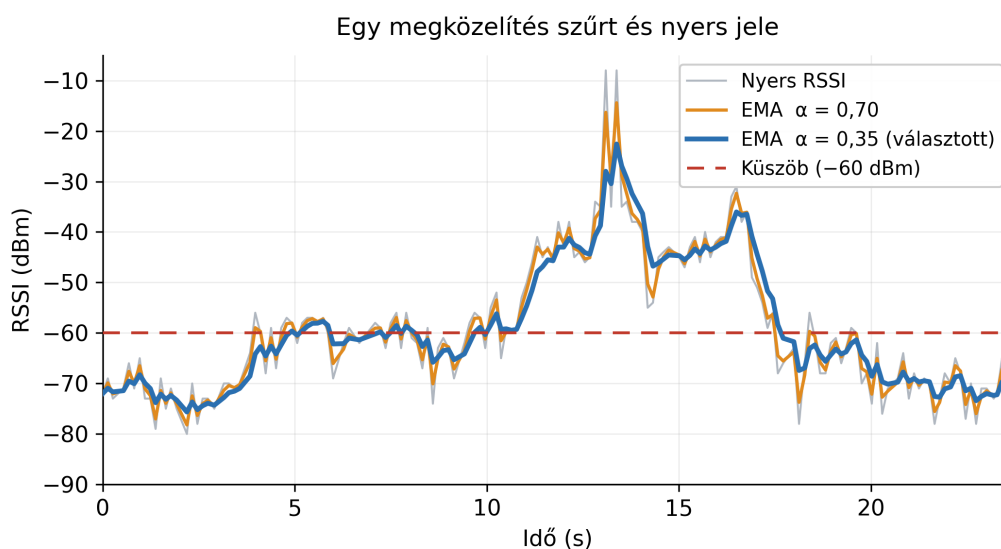
A szűrés hatása a küszöb helyzetétől is függ. A nyers jel téves pozitívjainak száma a -60 dBm-es küszöbnél a legnagyobb (19), a szigorúbb küszöböknél viszont lecsökken (mindkét esetben 1). Ennek oka, hogy a -60 dBm-es küszöb épp a zajos jelsáv közepébe esik, ahol a nyers RSSI sűrűn billeg a küszöb körül; a -50 dBm-es küszöb távolabb van a jellemző zajszinttől, így kevesebb indokolatlan átlépés adódik. A választott EMA-szűrő mindhárom küszöbnél nullára vitte a téves pozitívakat, a 5.2. táblázat szerint. Látható az is, hogy a küszöb szigorításával az átlépési idő közelít a nullához, vagyis a zár egyre inkább pontosan az ajtóhoz érkezéskor nyit, nem korábban.

A szűrőkön belül megfigyelhető a zajcsökkentés és a reakcióidő közötti átváltás. Az erős simítás (például a kis α -jú EMA vagy az alacsony Q -jú Kálmán-szűrő) nagyon sima jelet ad, de *korábban* lépi át a küszöböt, mert lassabban követi a valódi közeledést. A gyenge simítás gyors

Küszöb	Nyers jel		EMA $\alpha = 0,35$	
	Átlépés (s)	Téves poz.	Átlépés (s)	Téves poz.
-60 dBm	-11,98	19	-10,62	0
-55 dBm	-9,86	1	-4,44	0
-50 dBm	-4,83	1	-3,74	0

5.2. táblázat. A nyers jel és a választott EMA-szűrő viselkedése a három vizsgált küszöbnél. A szigorúbb küszöb a nullához közelíti az átlépési időt (a zár pontosabban az ajtónál nyit), és a nyers jel téves pozitívjai is csökkennek.

sabban reagál, viszont a maradék zaj miatt hajlamosabb a téves jelzésre (lásd az aszimmetrikus EMA agresszív beállításait, ahol a téves pozitívok száma újra megnő).



8. ábra. A nyers és a szűrt RSSI-jel egy jellemző megközelítés során. A választott EMA ($\alpha = 0,35$) simán követi a közeledést, és a -60 dBm-es küszöböt határozottan lépi át, míg a gyenge simítás ($\alpha = 0,70$) zajosabb marad.

Az adatok alapján a rendszerben használt **EMA** $\alpha = 0,35$ jó kompromisszum: nulla téves pozitívot ad, közepes simasággal, és a reakcióideje a küszöbtől függően a használathoz elegendő. A 2. fejezetben hozott tervezési döntést (EMA-szűrő alkalmazása) tehát a mérés is alátámasztja.

5.3. Az éles rendszer használati mérése

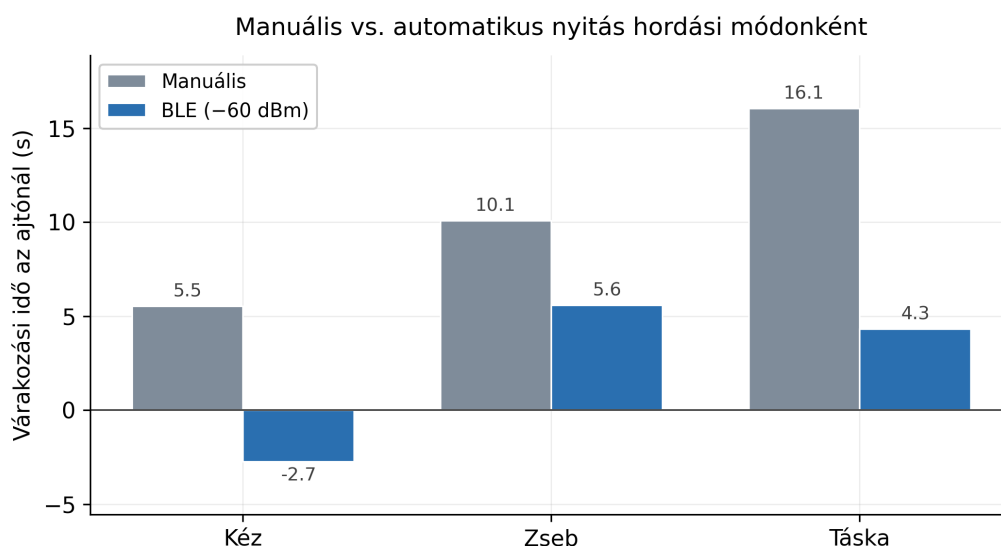
5.3.1. Nyitási sebesség: manuális vs. automatikus

A manuális nyitás az automatikus nyitás referenciapontja: azt mutatja meg, mennyi időt igényel a hozzáférés, ha a felhasználónak magának kell kezelnie a telefont. A manuális (alkalmazásból indított) nyitásnál a felhasználónak elő kell vennie a telefont, ezért a várakozási idő erősen függ attól, hol tartja a készüléket. A mérés szerint a kézben tartott telefonnal átlagosan 5,55 s, zsebből 10,09 s, táskából pedig 16,07 s telt el az ajtóhoz érkezéstől a bejutásig.

A BLE alapú automatikus nyitásnál a várakozási időt az ajtóhoz érkezés pillanatához viszonyítva mértem: a *negatív* érték azt jelenti, hogy a zár már nyitva volt, mire a felhasználó odaért. A -60 dBm-es küszöbnél a kézben tartott telefon esetén az átlag $-2,73$ s (vagyis a zár átlagosan jóval az ajtóhozérés előtt kinyílt), zsebből $+5,59$ s, táskából $+4,32$ s. A küszöb szigorításával (-55 , majd -50 dBm) a nyitás később következik be, mert a telefonnak közelebb kell érnie. A teljes, küszöb és hordási mód szerinti bontást a 5.3. táblázat tartalmazza.

Küszöb	Várakozási idő az ajtónál (s)		
	Kéz	Zseb	Táska
-60 dBm	$-2,73 \pm 1,42$	$5,59 \pm 2,54$	$4,32 \pm 4,01$
-55 dBm	$-1,09 \pm 2,72$	$6,41 \pm 3,82$	$11,36 \pm 3,86$
-50 dBm	$2,15 \pm 5,31$	$10,04 \pm 3,25$	$15,00 \pm 0,00$

5.3. táblázat. A BLE alapú automatikus nyitás várakozási ideje (átlag $\pm$ szórás) küszöbönként és hordási módonként. A negatív érték azt jelenti, hogy a zár már az ajtóhoz érkezés előtt kinyílt.



9. ábra. A manuális és a BLE alapú (-60 dBm) nyitás várakozási ideje hordási módonként. A kézben tartott telefon negatív értéke azt jelzi, hogy a zár már az ajtóhozérés előtt kinyílt.

A szórásértékek a nyitás *kiszámíthatóságáról* árulkodnak. A -60 dBm-es küszöbnél a kézben tartott telefon nemcsak a leggyorsabb, hanem a legkiszámíthatóbb is (szórás $1,42$ s); a zsebben ($2,54$ s), majd a táskában ($4,01$ s) tartott telefontól a szórás fokozatosan nő. Ez egybehangzik a jelgyengülés okával: minél jobban takarja a testet vagy az anyag a telefont, annál zajosabb a vett jel, és annál bizonytalanabb, hogy a szűrt érték mikor lépi át a küszöböt.

A testen elnyelődő jel hatása jól látszik: a kézben tartott telefon (szabad rálátás) érezhetően korábban nyit, mint a zsebben vagy táskában lévő, ahol a test, illetve az anyag gyengíti a jelet. A -60 dBm-es küszöbnél a zseb és a táska közötti különbség a mérési szóráson belül van, ezért itt a sorrendjük felcserélődhet; a hordási mód egyértelmű hatása a szigorúbb küszöböknel (-55 , -50 dBm) válik kivehetővé, ahol a táska adja a leghosszabb várakozási időt.

Külön figyelmet érdemel, hogy a táska módban a -50 dBm-es szigorú küszöbnél minden mérés pontosan $15,00$ s lett. Ez nem a kiszámíthatóság jele, hanem a mérési időablak felső

korlátja: ezekben az esetekben a BLE-nyitás a 15 másodperces határidőn belül egyáltalán nem következett be. A szigorú küszöb tehát a leginkább árnyékolt hordási módnál gyakorlatilag működésképtelenné teszi az automatikus nyitást, mert a telefonnak olyan közel kellene érnie az ajtóhoz, amilyen közel a táskában lévő, gyengített jel már nem jut el. Ez is a küszöb és a hordási mód együttes hatását mutatja: nincs olyan szigorú küszöb, amely minden hordási módnál egyformán jól működne.

5.3.2. Téves nyitások vizsgálata

A téves nyitás (false positive) azt jelenti, hogy a zár akkor is kinyit, amikor a jogosult felhasználó nem kívülről közeledik, hanem például belül, az ajtó közelében tartózkodik. Ez a 2. fejezetben tárgyalt elvi korlát közvetlen következménye: az RSSI nem hordoz irányinformációt. A mérés ezt számszerűsíti: a kintről érkező, jogosult bejutásnál a rendszer megbízhatóan nyitott (a -60 dBm-es küszöbnél mind a 168 kísérlet sikeres volt), a belső, 3 méteres FP-tesztben azonban a laza (-60 dBm) küszöbnél 160 esetből 27 alkalommal indokolatlanul kinyitott.

A küszöb szigorítása csökkenti a téves nyitásokat, de egyúttal a kintről történő sikeres bejutást is rontja: -50 dBm-nél a téves nyitások száma alacsonyabb, de a kinti bejutási kísérleteknek már csak egy része sikerült elsőre. A teljes összevetést a 5.4. táblázat mutatja.

Küszöb	Kinti bejutás (sikeres/összes)	Benti téves nyitás (FP)
-60 dBm	168/168	27
-55 dBm	149/168	13
-50 dBm	96/168	16

5.4. táblázat. A nyitási küszöb hatása: a kintről érkező jogosult bejutás sikeressége és a belső, 3 méteres tesztben mért téves nyitások száma. A laza küszöb megbízhatóan beenged, de sok téves nyitást ad; a szigorítás csökkenti a téves nyitásokat, de a sikeres bejutást is rontja.

Ez az eredmény mutatja a küszöb megválasztásának kompromisszumos jellegét, és egyúttal indokolja a 2. fejezetben tett tervezési döntést, hogy a küszöb a felhasználó által testre szabható paraméter legyen.

5.3.3. Hosszú távú stabilitás

A rendszer folyamatos működését többnapos megfigyeléssel vizsgáltam. A 5.5. táblázat a napi állapotot foglalja össze. A megfigyelt napokból a rendszer többségében stabilan, beavatkozás nélkül működött; a néhány instabil napon a hibák jellemzően a mérési referencia-eszközzel való ESP-NOW kapcsolatból (ACK-hiba), illetve hálózati időtúllépésből eredtek, és a rendszer az automatikus újraindítással helyreállt.

5.3.4. A mérés korlátai

A mérési eredmények értelmezésekor néhány korlátot figyelembe kell venni. A használati mérés egyetlen telefonkészülékkel és egy adott lakásban készült, ezért az abszolút értékek (a konkrét várakozási idők és a küszöbönkénti viselkedés) környezet- és eszközfüggők: más antennájú

Dátum	Állapot	Újraind.	Megjegyzés
2026-04-23	Instabil	0	ESP-NOW ACK-hiba
2026-04-24	Instabil	1	Hálózati időtúllépés
2026-04-25	Stabil	0	—
2026-04-26	Stabil	0	—
2026-04-27	Stabil	0	—
2026-04-28	Stabil	0	—
2026-04-29	Stabil	0	—
2026-04-30	Stabil	0	—
2026-05-03	Stabil	0	—
2026-05-04	Instabil	0	ESP-NOW ACK-hiba
2026-05-05	Stabil	0	—
2026-05-06	Instabil	2	Hálózati időtúllépés
2026-05-07	Stabil	0	—
2026-05-08	Stabil	0	—
2026-05-09	Stabil	0	—
2026-05-10	Stabil	0	—

5.5. táblázat. A rendszer napi stabilitási naplója a megfigyelt időszakban. A 16 napból 12 teljesen stabil volt; az instabil napokon a hibák a mérési referencia-eszköz ESP-NOW kapcsolatából, illetve hálózati időtúllépésből eredtek.

készülék, eltérő falazat vagy bútorzat más jelviszonyokat adhat. A mért *tendenciák* azonban — a szűrés szükségessége, a hordási mód hatása, az irányérzéketlenségből eredő téves nyitások, a küszöb kompromisszumos jellege — a BLE-jel fizikai természetéből következnek, így általánosíthatók. Az offline szűrő-összehasonlítás ezzel szemben rögzített, azonos adatsoron futott, ezért az eljárások egymáshoz viszonyított sorrendje ettől a környezeti függéstől mentes.

5.4. Az eredmények értékelése

A mérések összességében alátámasztják a 2. fejezetben hozott tervezési döntéseket, ugyanakkor élesen megmutatják a rendszer korlátait is.

A szűrés szükségessége egyértelműen igazolódott. A nyers jel a -60 dBm-es küszöbnél 19 téves pozitívot adott, miközben a szűrt változatok ezt nullára vagy közel nullára szorították. Ez megerősíti, hogy az RSSI önmagában, feldolgozatlanul nem alkalmas megbízható nyitási döntésre. A szűrőcsaládok összevetése a zajcsökkentés és a reakcióidő közötti átváltást is jól láthattóvá tette: az erős simítás (kis α , alacsony Q) ugyan kevés téves pozitívot ad, de a jel lassabban követi a valódi közeledést. A választott EMA $\alpha = 0,35$ ezen a skálán jó középutnak bizonyult: nulla téves pozitív, kezelhető simaság, és a használathoz elegendő reakcióidő. A Kálmán-szűrő hasonlóan jó eredményt adott, de számítási igénye és a két hangolando paramétere (Q , R) miatt a beágyazott környezetben az EMA egyszerűsége indokolt választás maradt.

A használati mérés a rendszer gyakorlati hasznát mutatta meg. A kézben tartott telefonnál a BLE alapú nyitás átlagosan az ajtóhoz érkezés előtt kinyitott ($-2,73$ s a -60 dBm-es küszöbnél), miközben ugyanitt a manuális nyitás $5,55$ s várakozást igényelt. A különbség a zsebben és táskában tartott telefonnál is megmaradt, bár ott a test és az anyag jelgyengítő hatása miatt kisebb. A mérés azt is megerősítette, hogy az RSSI hordási módtól való függése nem elha-

nyagolható: ugyanaz a küszöb a három hordási módnál eltérő nyitási időt ad, ami közvetlenül indokolja a küszöb testreszabhatóságát.

A rendszer fő korlátja az irányérzéketlenség, amely a téves nyitások mérésében számszerűsödött. Mivel az RSSI nem hordoz irányinformációt, a belső térben, az ajtó közelében tartózkodó jogosult telefon is nyitást válthat ki. A küszöb szigorítása csökkenti ezt, de a kintről történő sikeres bejutás rovására: a -60 dBm-es küszöbnél a kinti bejutás hibátlan (168/168), -50 dBm-nél viszont már csak az esetek mintegy 57%-a sikerült elsőre. Érdemes megjegyezni, hogy a belső téves nyitások száma nem csökkent szigorúan monotonon a küszöbvel (a -55 dBm-es 13 után a -50 dBm-es 16 értéket adott); ez arra utal, hogy a 3 méteres belső pont az átmeneti tartományba esik, ahol a környezeti ingadozás is befolyásolja az eredményt. A küszöb megválasztása tehát minden esetben kompromisszum a kényelmes bejutás és a téves nyitások elkerülése között, és nincs olyan egyetlen érték, amely mindkettőt egyszerre optimalizálná.

A többnapos stabilitási mérés szerint a rendszer a megfigyelt 16 napból 12-n beavatkozás nélkül, stabilan működött. Az instabil napokon a hibák a mérési referencia-eszköz ESP-NOW kapcsolatából, illetve hálózati időtúllépésből eredtek, nem magából a zárvezérlő logikából; a rendszer az automatikus újraindítással minden esetben helyreállt. Ez a hibatűrés egy önállóan, felügyelet nélkül működő beléptető eszköznél lényeges tulajdonság.

6. fejezet

Jövőbeli fejlesztések lehetősége és projekt összegzés

Könyvészet

- [1] Espressif Systems, *ESP32-S3-WROOM-1 & ESP32-S3-WROOM-1U Datasheet*, 2023, v1.8. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf
- [2] Texas Instruments, *LM2596 SIMPLE SWITCHER Power Converter 150-kHz 3-A Step-Down Voltage Regulator*, 2023, datasheet, SNVS124G. [Online]. Available: <https://www.ti.com/lit/ds/symlink/lm2596.pdf>
- [3] P. Horowitz and W. Hill, *The Art of Electronics*, 3rd ed. Cambridge University Press, 2015.
- [4] Vishay General Semiconductor, *1N4001 thru 1N4007 – General Purpose Plastic Rectifier*, 2020, datasheet, Document Number 88503. [Online]. Available: <https://www.vishay.com/docs/88503/1n4001.pdf>
- [5] Bluetooth SIG, *Bluetooth Core Specification*, 2021, version 5.3.
- [6] R. Faragher and R. Harle, „Location fingerprinting with bluetooth low energy beacons,” *IEEE Journal on Selected Areas in Communications*, vol. 33, no. 11, pp. 2418–2428, 2015.
- [7] Apache Software Foundation, „Apache Mynewt NimBLE User Guide,” <https://mynewt.apache.org/latest/network/ble/>, hozzáférés: 2026.
- [8] Google, „Firebase Authentication Documentation,” <https://firebase.google.com/docs/auth>, hozzáférés: 2026.
- [9] —, „Firebase Realtime Database Documentation,” <https://firebase.google.com/docs/database>, hozzáférés: 2026.
- [10] JetBrains, „Kotlin Programming Language Documentation,” <https://kotlinlang.org/docs/home.html>, hozzáférés: 2026.
- [11] Google, „Android Developers – Guide to App Architecture,” <https://developer.android.com/topic/architecture>, hozzáférés: 2026.

A. függelék

Mellékletek